Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Ставьте “+” в чат, если да
Напишите, если есть проблемы

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Конкурентность с Sequence и Flow-потоками

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитек- тура		
Модуль 4: Разработка API	OpenAPI	CollabAPI	Модели и маперы	Testing	Monitoring	
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози- торий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Маршрут вебинара

- Sequence
- Flow
- Расширения Flow: StateFlow, SharedFlow
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Научиться использовать Sequence
2. Научиться использовать Flow

Смысл

Зачем вам это уметь

1. Для владения навыками реактивного программированния
2. Чтобы писать эффективный асинхронный и многопоточный код для обработки потоков данных
3. Чтобы легко использовать различные API совместно с каналами и потоками

Задание

Что такое коллекции?

Какие коллекции известны вам?

Что представляет интерфейс Iterable<T>?

Sequence

Sequence<T>

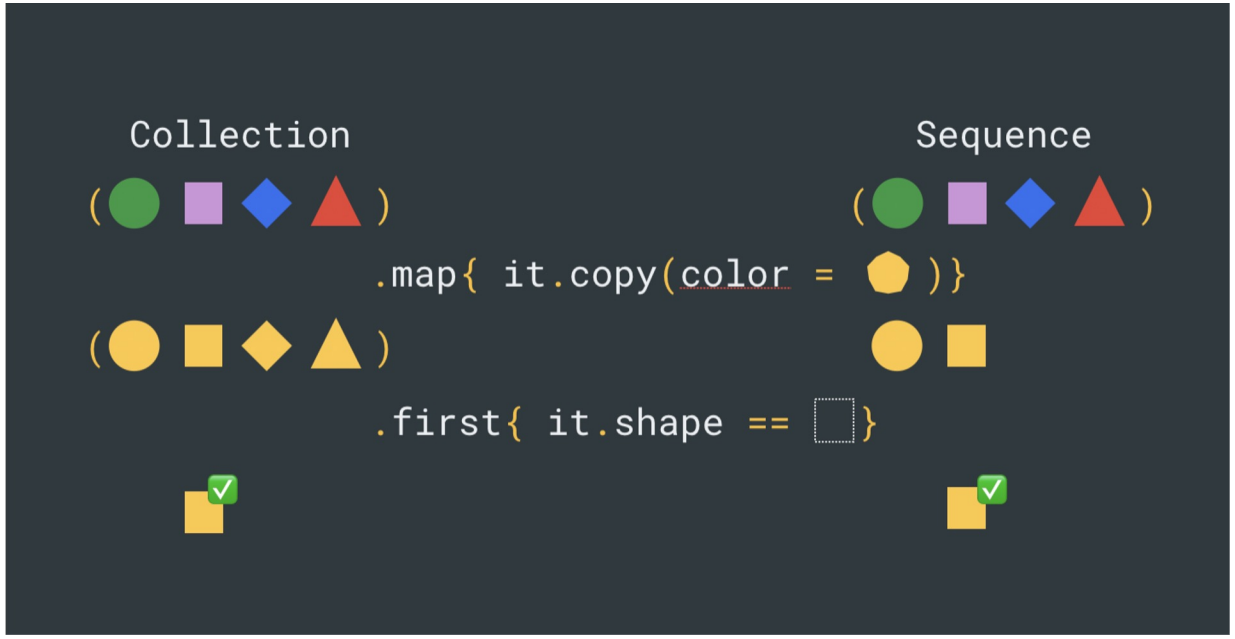
```
public interface Sequence<out T> {
   /**
    * Returns an [Iterator] that returns the values from the sequence.
    *
    * Throws an exception if the sequence is constrained to be iterated once and `iterator` is
invoked the second time.
    */
   public operator fun iterator(): Iterator<T>
}
```

- Аналогичен другим итерируемым
- Представляет “ленивую” последовательность элементов

Отличие Sequence<T> и Collection<T>

```
val figure = listOf<Figure>(...)
    .map { it.copy(color = Color.YELLOW) }
    .first { it.shape == Shape.SQUARE }
```

```
val figure = sequenceOf<Figure>(...)
    .map { it.copy(color = Color.YELLOW) }
    .first { it.shape == Shape.SQUARE }
```



- Операторы Sequence<T> вычисляют каждый элемент по запросу
- Операторы Collection<T> вычисляют все элементы разом

Создание Sequence<T>

```
// Пустая последовательность
emptySequence()

// Из набора элементов
sequenceOf(1, 2, 3)

// Ленивый билдер с suspend-лямбдой
sequence {
   yield(1)
   Thread.sleep(1000)
   yield(2)
   yield(3)
}

// Из коллекции
listOf(1, 2, 3).asSequence()
```

```
// Используя лямбда-фабрику итераторов
Sequence { iterable.iterator() }

// Используя функции для генерации значений
generateSequence { Random.nextInt() }

generateSequence(1) { it + 1 }

generateSequence(
    { Random.nextInt() },
    { it + 1 }
)
```

Основные выводы

- **Sequence<E>** представляет “ленивую” последовательность элементов
- Элементы последовательности вычисляются только по запросу
- Эффективны для больших/бесконечных последовательностей
- Эффективны для больших цепочек операторов
- Подходит для последовательностей отдающих элементы с задержкой
- Вызовы могут заблокировать поток

LIVE

Flow

Flow<T>

```
public interface Flow<out T> {

   @InternalCoroutinesApi
   public suspend fun collect(collector: FlowCollector<T>)
}

public interface FlowCollector<in T> {
   /**
    * Collects the value emitted by the upstream.
    * This method is not thread-safe and should not be invoked concurrently.
    */
   public suspend fun emit(value: T)
}
```

- Представляет “ленивую” последовательность элементов, аналогично Sequence<T>
- Исполняется в корутинах

Этапы при работе с flow-потоками

```
flow<Int> { emit(...) }

    .onEach { println("Emitted: $it") }

    .map { it + 1 }
    .filter { it % 2 == 0 }

    .collect { println("Collected: $it") }
```

1. Билдер

2. Операторы
   - side-эффекты
   - трансформации

3. Терминальный оператор

Потоки и корутины

```
flow<Int> { emit(...) }

    .onEach { println("Emitted: $it") }

    .map { it + 1 }
    .filter { it % 2 == 0 }

    .collect { println("Collected: $it") }
```

- Вычисления запускаются при вызове терминального оператора
- Вычисления происходят в контексте корутины запустившей терминальный оператор
- Терминальный оператор прерывает запустившую корутину

Смена контекста

```
withContext(Dispatchers.Main) {
   val singleValue = intFlow // будет выполнен в IO, если другое не указано в потоке раньше
       .map { ... } // будет выполнен в IO
       .flowOn(Dispatchers.IO)
       .filter { ... } // будет выполнен в Default
       .flowOn(Dispatchers.Default)
       .single() // будет выполнен в Main
}
```

- Оператор flowOn запускает upstream операторы в указанном контексте
- Для связи upstream и downstream используется канал
- flowOn действует в ОБРАТНОМ ПОРЯДКЕ, т.е. на все операторы до него

Flow is cold

```
val flow = flow<Int> { println("Starter") }

launch { flow.collect() }

launch { flow.collect() }

// console output:
// Started
// Started
```

- Каждый вызов терминального оператора запускает поток заново

Билдеры

```
// Пустой поток
emptyFlow()

// Из набора элементов
flowOf(1, 2, 3)

// Ленивый билдер
flow {
   emit(1)
   emit(2)
   delay(1000)
   emit(3)
}

// Из коллекции
listOf(1, 2, 3).asFlow()
```

```
// Из последовательности
sequenceOf(1, 2, 3).asFlow()

// Контекстно- и потоко-безопасный билдер
channelFlow {
   launch { send("foo") }
   delay(100)
   send("bar")
}

// Контекстно- и потоко-безопасный билдер
callbackFlow {
   val timer = Timer()
   timer.scheduleAtFixedRate(6L, 1000L) {
       trySendBlocking("text")
   }
   awaitClose { timer.cancel() }
}
```

Билдеры

Билдер	Описание	Использование
emptyFlow()	Пустой поток	Когда данных нет, и нужно вернуть поток-заглушку
flowOf(...)	Из фиксированного набора элементов	Быстрая генерация данных из небольшого количества значений
flow {}	Ленивый поток с произвольной логикой.	Для генерации данных на основе сложной логики или асинхронных операций
asFlow()	Преобразование коллекции/последовательности	Когда данные уже в коллекции или последовательности
channelFlow {}	Потокобезопасный билдер с несколькими корутинами	Для параллельной генерации значений из нескольких корутин
callbackFlow {}	Поток из обратных вызовов	Для работы с асинхронными API или событиями пользовательского интерфейса

Операторы

```
// side-эффект
flow.onEach { log.info("Got item: $it") }

// side-эффекты/трансформации
flow.onStart { emit(...) }
flow.onCompletion { emit(...) }
flow.onEmpty { emit(...) }
flow.catch { emit(...) }

// трансформации
flow.map { it + 1 }
flow.filter { it != "foo" }
flow.take(10)
flow.drop(5)
```

```
// буферизация
flow.buffer(128, BufferOverflow.DROP_OLDEST)
flow.conflate()

// throttling
flow.debounce(1000)
flow.sample(2000)

// retry
flow.retry(5) { throwable -> true }
flow.retryWhen { throwable, tryNum -> true }

// смена контекста выполнения
flow.flowOn(Dispatchers.IO)
```

Операторы

Категория	Операторы	Описание
Побочные эффекты	onEach, onStart, onCompletion, onEmpty, catch	Выполняют действия (логирование, обработка ошибок).
Трансформации	map, filter, take, drop	Преобразуют, фильтруют или ограничивают поток.
Буферизация	buffer, conflate	Сглаживают разницу в скорости между производителем и потребителем.
Throttling	debounce, sample	Управляют частотой обработки событий.
Повторные попытки	retry, retryWhen	Повторяют выполнение потока при ошибках.
Контекст выполнения	flowOn	Меняет контекст выполнения операторов.

Как написать оператор?

```
fun <T: Any> Flow<T>.zipWithNext(): Flow<Pair<T, T>> = flow {
   var current: T? = null
   collect { next ->
       current?.also { emit(it to next) }
       current = next
   }
}

flowOf(1, 2, 3, 4)
   .zipWithNext()
   .collect { println(it) }

// console output:
// (1, 2)
// (2, 3)
// (3, 4)
```

Терминальные операторы

```
flow.collect { println(it) }

flow.collect()

val item = flow.first()

val item = flow.first { true }

val item = flow.single()

flow.launchIn(coroutineScope) // аналогично coroutineScope.launch { flow.collect() }

val items = flow.toList()
```

Терминальные операторы

Оператор	Описание
collect {}	Запускает поток и обрабатывает каждый элемент с помощью переданного обработчика
collect()	Запускает поток без дополнительной обработки каждого элемента
first()	Возвращает первый элемент потока (или выбрасывает исключение, если поток пуст)
first {}	Возвращает первый элемент, удовлетворяющий переданному условию
single()	Возвращает единственный элемент потока (или выбрасывает исключение, если элементов больше или поток пуст)
launchIn(coroutineScope)	Запускает поток в корутине с указанным контекстом
toList()	Собирает все элементы потока в список

Основные выводы

- **Flow<E>** представляет асинхронную последовательность элементов
- Этапы работы: билдер, операторы, терминальный оператор
- Flow is cold
- Все вычисления происходят в контексте корутины запустившей поток
- Для смены контекста используется flowOn оператор
- Отличие Flow от Sequence: Flow Работает в асинхронном режиме, основываясь на корутинах, а Sequence работает в синхронном

LIVE

Расширения Flow: StateFlow, SharedFlow

SharedFlow

- SharedFlow is hot
- Обеспечивает доставку сообщений всем подписчикам одновременно

```
val shFlow = MutableSharedFlow<String>()
val j1 = launch { shFlow.collect { println("XX $it") } }
val j2 = launch { shFlow.collect { println("YY $it") } }
```

StateFlow

- StateFlow is hot
- Обеспечивает управление состояниями:
  - Хранение
  - Синхронизация
  - Оповещение

```
val mshState = MutableStateFlow("state1")
println("FINAL STATE: ${mshState.value}")
```

LIVE

Вопросы по теме

1. В чем разница между каналами и последовательностями?
2. Чем отличаются каналы и последовательности от коллекций
3. Какие особенные flow знаете?

15 минут

Ответы пишем в чат

Тезисы

Подведем итоги

1. Sequence и Flow обеспечивают ленивую обработку данных
2. Sequence работает в синхронном контексте, Flow – в корутин-контексте
3. Жизненный цикл Sequence и Flow: билдер, операции обработки, терминальный оператор
4. Типичные операторы: onEach, map, reduce, sample, ...; терминальные операторы: collect, first, single
5. Расширения Flow – SharedFlow, StateFlow – горячие реализации

Слайд с домашним заданием

1. Создать три композитных проекта: (1) учебные модули; (2) плагин; (3) проектные модули
2. Создать плагины Gradle для сборки подпроектов (1) на базе jvm; (2) на базе multiplatform
3. В плагины добавить такси для генерации докер-образов
4. Создать файл libs.versions.toml и зацепить за него версии зависимостей в плагинах

Сроки выполнения: до начала Модуля 3

Рефлексия

Цели вебинара

Проверка достижения целей

1. Научиться использовать Sequence
2. Научиться использовать Flow

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202411-otuskotlin-marketplace
- https://kotlinlang.org/docs/sequences.html
- https://kotlinlang.org/docs/flow.html
- https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-shared-flow/
- https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-state-flow/

Следующий вебинар

Модуль 2: Расширенные возможности Kotlin

Мультиплатформенная разработка

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары